A Provably Secure Certificateless Signature Scheme With Anonymity for Healthcare IIoT

Zirui Qiao, Ran Xu, Yanwei Zhou, Bo Yang, Tianqing Zhu, and Mingwu Zhang

***Abstract*—The Industrial Internet of Things (IIoT) is changing our way of life and work. As the number of mobile devices connected to IIoT increases, users will face many security challenges, such as insecure communication environments. The certificateless signature (CLS) scheme can ensure the integrity and validity of data and provide secure identity authentication for the IIoT, and several pairing-free CLS schemes have been proposed in recent years. However, we find they are vulnerable to the signature forgery attack of malicious key generation centers by safety analysis, which does not realize the security they have claimed. To solve this problem, we show an improved CLS scheme that achieves anonymity and formally proves its security under the hardness of the discrete logarithm problem in the random oracle. Comprehensive performance analysis and comparison show that our scheme has less communication and computation costs with higher security, and our construction is suitable for scenarios with limited resources. Finally, we apply this scheme to construct a mutual identity authentication protocol in the healthcare IIoT environment.**

***Index Terms*—Authentication, certificateless signature (CLS), healthcare Industrial Internet of Things (IIoT), provable security.**

I. Introduction

IN RECENT years, the success of the Internet of Things (IoT) has recently spread to the industrial sector, connecting previously isolated components and the Internet through an open and global industrial network platform. This trend is commonly known as the Industrial IoT (IIoT). IIoT with its characteristics of comprehensive perception, intelligent processing, self-organization, and self-maintenance, has caused a wave of change "represented by the IIoT" around the world [1]. IIoT has greatly improved the production efficiency of industrial agriculture, medical, and health care, etc. In the medical field, the health IIoT (health IIoT) has greatly promoted the intelligence and informatization of the medical system by monitoring and storing medical information and providing timely and effective medical diagnosis services. The medical research team obtains health data, such as blood oxygen, blood pressure, heart rate by developing micro medical sensors, and placing sensor nodes on the patient's body. Since devices have limited computing and storage capabilities, and the sensor data is growing exponentially, the cloud server is needed to store data. However, there are serious security problems in data outsourcing, patients' health data are vulnerable to malicious attacks during transmission and sharing. The disclosure and forgery of health information damage patients' privacy, and even affect the diagnosis results of doctors, threatening life health, and leading to the deterioration of doctor-patient relationships. Therefore, it is very significant to ensure the validity and security of data transmission. Digital signatures can provide identity authentication and protect the confidentiality and integrity of data [2], [3], [4].

Certificate management problem has existed in traditional public key infrastructure. Although this problem can be solved by identity-based public key cryptography, there is an inherent key escrow problem in the this system. A cryptosystem called the certificateless public key cryptography (CL-PKC) is proposed in [5]. In CL-PKC, the user's complete private key is generated by user and the key generation center (KGC) where user chooses the secret value and KGC generates the user's partial private key. This cryptosystem does not need digital certificates and overcomes certificate management problem. In addition, KGC only has part of the private key, which solves the key escrow problem. To further describe the security, Girault [6] defined three security levels for a trusted third party, where level 3 means the strongest security for the system. Chen et al. [7], proposed three types of adversaries in the CLS scheme and proved their construction has achieved Girault's level 3 security. Karati et al. [8] reached Girault's level 3 security in their certificateless encryption scheme.

A. Related Works

Because bilinear mapping is time-consuming [9], [10], [11], [12], [13], some pairing-free certificateless signature (CLS) schemes were proposed to improve computational efficiency. Jia et al. [14] created a construction of CLS scheme, but Du et al. [15] and Xiang et al. [16] proved that

Received 8 February 2025; revised 15 March 2025; accepted 17 March 2025. Date of publication 24 March 2025; date of current version 27 June 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62272287, Grant U24B20149, and Grant 62472150; in part by the Key Research and Development Project in Shaanxi Province under Grant 2024GX-YBXM-074; in part by the Research Fund for Key Laboratory of Data Protection and Intelligent Management, Ministry of Education, Sichuan University under Grant SCUSAKFKT202401Y; and in part by the Fundamental Research Funds for the Central Universities under Grant GK202301009. *(Corresponding authors: Yanwei Zhou; Ran Xu.)*

Zirui Qiao is with the School of Cyberspace Security, Xi'an University of Posts and Telecommunications, Xi'an 710121, China, and also with the Key Laboratory of Data Protection and Intelligent Management, Ministry of Education, Sichuan University, Chengdu 610065, China.

Ran Xu, Yanwei Zhou, and Bo Yang are with the School of Computer Science, Shaanxi Normal University, Xi'an 710062, China (e-mail: xr10220914@163.com; zyw@snnu.edu.cn).

Tianqing Zhu is with the Faculty of Data Science, City University of Macau, Macau, China.

Mingwu Zhang is with the School of Computer, Hubei University of Technology, Wuhan 430068, China.

Digital Object Identifier 10.1109/JIOT.2025.3554188

2327-4662 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 14,2025 at 03:49:37 UTC from IEEE Xplore. Restrictions apply.

it is vulnerable to Type-I adversary. Ma et al. [17] found the above two schemes are both easily attacked by Type-I adversaries. Wang et al. [18] proposed a CLS scheme for resource-limited systems, and the computation cost is too high. Thumbur et al. [19] showed a CLS scheme and proved its unforgeability. Xu et al. [20] noted that the above scheme will be broken by the forgery attack of Type-I adversaries. They constructed a new scheme and then proved the security of this scheme. Recently, Wang et al. [21] illustrated Xu et al.'s scheme [20] is vulnerable to KGC compromised and replay attacks. They introduced a decentralized smart contract executing on the Ethereum platform to replace the traditional centralized KGC. However, we find the above scheme [21] cannot resist the attack of Type-II adversary and does not achieve the intended purpose. Liu et al. [22] proposed a new certificateless blind signature scheme achieving conditional revocation function. Shim [23] showed a new secure CLS scheme for cloud-assisted IIoT. However, the above CLS schemes only achieved Girault's level 2 security, Shim's scheme [24] and Yang et al.'s scheme [25] have achieved the Girault's level 3 security, but they did not give a specific security proof.

In the past few years, many CLS schemes [26], [27], [28] have been proposed. However, some of these are based on bilinear mapping and have low computational efficiency, and some have security defects and do not meet the security requirements. Therefore, we propose a new CLS scheme and prove that it can resist the attack of Type-I, Type-II, and Type-III adversaries achieving the Girault's level 3 security. Through performance analysis, it is shown that our CLS scheme has lower communication and computing costs.

B. Our Motivations

Without loss of generality, researchers create some CLS schemes to ensure the validity of communication data. However, from the above description, we can find that many constructions do not meet the required security requirements for signature forgery attacks. In this article, to further solve the above problems, we provide the correct method for building a secure CLS solution to avoid such security issues in the future.

In health IIoT, when doctors and patients send messages to each other, they need to confirm the legitimacy of each other's identities, so a mutual identity authentication protocol should be employed to ensure that both parties are legitimate entities. At the same time, to protect privacy in the communication process, the corresponding protocol has realized anonymity. Furthermore, the devices typically have limited computing and storage capabilities, and the corresponding protocol needs to provide highly efficient computing and communication, and we can use the proposal in resource-constrained environments.

As discussed above, in this article, we first construct a novel CLS scheme and establish its formal security proof under the random oracle model. Subsequently, tailored to the pragmatic requirements of the health IIoT, we propose a lightweight identity authentication protocol based on the aforementioned CLS scheme, specifically designed to address the critical challenge of identity legitimacy verification between patients and physicians within health IIoT.

TABLE I
LIST OF SYMBOL DEFINITION

Symbol	Definition	Symbol	Definition
G	addition cyclic group	Z_q^*	finite field
κ	security parameter	msk	master secret key
$params$	system public parameter	RID	real identity
ID	temporary identity	$\mathcal{A}_1$	Type I adversary
PK	public key	$\mathcal{A}_2$	Type II adversary
SK	private key	$\mathcal{A}_3$	Type III adversary

C. Our Contributions

In this article, we focus on the security of data transmission in healthcare IIoT by designing a provably secure CLS scheme and constructing a mutual identity authentication protocol in the medical environment based on our CLS scheme, which protects privacy information for both sides of the communication through anonymity. We summarize the main specific contributions as the following.

1) We create a novel CLS scheme without pairing operations and then prove it has achieved Girault's level 3 security. That is, our proposal has attached the highest security level. Furthermore, a comprehensive comparison of the corresponding CLS schemes is employed to prove that the above construction has less communication and computation costs with higher security and also shows that it is suitable for resource-constrained scenarios of health IIoT.
2) We propose a mutual identity authentication and key agreement protocol utilizing our CLS scheme, which realizes identity authentication between doctors and patients in the actual health IIoT environment and proves that the above protocol has better security and performance. We conduct simulation experiments to integrate our scheme with the healthcare system, demonstrating our protocol applied in the healthcare system by experiment.

The proposed CLS scheme demonstrates strong unforgeability and ensures anonymity in the healthcare health IIoT context. It effectively maintains the integrity of patient health information while preventing the leakage of sensitive privacy data through cryptographic safeguards.

II. PRELIMINARIES

The symbols used in this article are shown in Table I.

A. Complexity Assumption

Definition 1 (Discrete Logarithm (DL) Assumption): Let G be an addition cyclic group with prime order q, and P is the generator of G. For any probabilistic polynomial time adversary, it is difficult to compute $a \in Z_q^*$ given $Q = aP$. That is, the advantage of the adversary $\mathcal{A}$ is described as $\mathsf{Adv}_{\mathcal{A}}^{\mathrm{DL}}(\kappa) = \Pr[\mathcal{A}(P, aP) = a]$.

The DL assumption shows that, for any adversary $\mathcal{A}$, the above advantage $\mathsf{Adv}_{\mathcal{A}}^{\mathrm{DL}}(\kappa)$ is negligible.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 14,2025 at 03:49:37 UTC from IEEE Xplore. Restrictions apply.

B. Girault's Security Level

In [6], Girault distinguished the authority into three trust levels. Then, Hu et al. [29] extended the above security level from the CLS scheme. The three levels are as follows.

1) *Level 1:* KGC masters users' private keys, who can forge the signature as an impersonated user.
2) *Level 2:* KGC does not know users' private keys, and it also can forge the signature by forging the false private key as an impersonated user.
3) *Level 3:* KGC does not know users' private keys, who cannot forge the signature by forging a false private key. The forged private keys for all users cannot be undetected.

C. Forking Lemma

The Forking Lemma demonstrates that in the security proof of CLS schemes under the random oracle model, the challenger employs hash function replay techniques to enable an adversary to generate two forged signatures based on the same random value but distinct hash responses. Specifically, if an adversary can produce a valid forged signature with advantage ε, it can then output two valid forged signatures with advantage $(1-[1/e])(\varepsilon/Q_h)$, where e denotes the base of the natural logarithm, and Q_h represents the number of random oracle queries [30].

III. Certificateless Signature Scheme

Now, we will review the formal definition and security model of the CLS scheme.

A. Formal Definition

A CLS scheme has the following algorithms.

Setup: KGC executes the setup algorithm given the security parameter κ and produces master secret key *msk* and parameters *params*. That is, $(params, msk) \leftarrow \mathsf{Setup}(1^\kappa)$.

Partial Private Key: KGC inputs the identity ID, *msk* and parameters *params*, and runs the algorithm, finally, KGC outputs the d as the partial private key to the user. That is, $d \leftarrow \mathsf{ParPriKey}(\mathrm{ID}, params, msk)$.

Secret Value: User executes this algorithm given the parameters *params* and ID, then returns x as the secret value. That is, $x \leftarrow \mathsf{SecVal}(params, \mathrm{ID})$.

Public/Private Key: User executes this algorithm given the parameters *params*, ID, d, and x as input, then outputs the public key and private key (PK, SK), respectively. That is, $(PK, SK) \leftarrow \mathsf{KeyGen}(params, \mathrm{ID}, d, x)$.

Signature: The signer inputs parameters *params*, message m, identity ID, and private key SK, then runs this algorithm and outputs the signature δ about the message m. That is, $\delta \leftarrow \mathsf{Sign}(params, \mathrm{ID}, SK, m)$.

Verify: The verifier takes the parameters *params*, ID, m, and δ as input, and runs the algorithm. Output "1" if the δ is valid; Otherwise, output "0." That is, $1/0 \leftarrow \mathsf{Verify}(params, \mathrm{ID}, m, \delta)$.

We have to stress that, in the design of the CLS scheme, it is standard practice to integrate the secret value generation and partial private key creation within a unified algorithmic framework.

B. Security Model

According to the different types of attacks on CLS schemes, we classify the adversaries into three types: Type I adversaries $\mathcal{A}_1$, Type II adversaries $\mathcal{A}_2$, and Type III adversaries $\mathcal{A}_3$. $\mathcal{A}_1$ initiates an external attack, which can perform public-key replacement attacks. However, $\mathcal{A}_1$ can not capture the user's partial private key and the master secret key. $\mathcal{A}_2$ initiates an internal attack, which is a malicious KGC and can start a master secret key attack but cannot replace the public key of users. $\mathcal{A}_3$ impersonates a legitimate user, which forges public and private keys that do not hold by itself.

The unforgeability of the CLS scheme requires that the corresponding scheme is unforgeable under the chosen message attack, and we define two games as the following to prove it.

Game 1: This game is executed between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}_1$, the specific execution process is as follows:

Initialization: $\mathcal{C}$ executes the setup algorithm Setup, and produces master secret key *msk* which is kept secretly and parameters *params*, then sends *params* to $\mathcal{A}_1$.

Queries: At this stage, $\mathcal{A}_1$ can perform the following inquiries of polynomial bounded degree. Furthermore, the following queries are performed adaptively.

1) *Creation-User:* When $\mathcal{A}_1$ asks the creation user query with the identity ID, $\mathcal{C}$ executes $\mathsf{ParPriKey}$ to create d, runs SecVal to create x, runs KeyGen to create (PK, SK), and returns PK to $\mathcal{A}_1$.
2) *Extract-Partial-Private-Key:* When $\mathcal{A}_1$ asks this query with ID, $\mathcal{C}$ executes $\mathsf{ParPriKey}$ and returns d to $\mathcal{A}_1$.
3) *Extract-Secret-Value:* When $\mathcal{A}_1$ asks this query with ID, $\mathcal{C}$ executes SecVal, and returns x to $\mathcal{A}_1$.
4) *Extract-Private-Key:* When $\mathcal{A}_1$ asks this query with ID, $\mathcal{C}$ executes $\mathsf{ParPriKey}$ to create d, runs SecVal to create x, runs KeyGen to create (PK, SK), and returns SK to $\mathcal{A}_1$.
5) *Replace-Public-Key:* When $\mathcal{C}$ receiving this query, use a new value PK^* to replace public key PK.
6) *Signature:* When $\mathcal{A}_1$ asks this query with (ID, m), $\mathcal{C}$ executes signature algorithm Sign to generate δ, and outputs it to $\mathcal{A}_1$.

Forgery: $\mathcal{A}_1$ outputs a forgery signature δ^* for the challenge identity message pair (ID^*, m^*), when the following conditions are all met, the adversary $\mathcal{A}_1$ wins the game:

1) $\mathcal{A}_1$ cannot ask *Extract-Partial-Private-Key* and *Extract-Private-Key* queries for ID^*.
2) For any identity of the replaced public key, $\mathcal{A}_1$ can not perform *Extract-Private-Key* query.
3) $\mathcal{A}_1$ can never ask a signature query for (ID^*, m^*), and δ^* is a valid signature.

Therefore, for any PPT adversary $\mathcal{A}_1$, the advantage of winning this game is $\mathsf{Adv}_{\mathcal{A}_1}(\kappa) = \Pr[\mathcal{A}_1 \ wins]$.

Game 2: This game is executed between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}_2$, the detailed execution process is following:

Initialization: $\mathcal{C}$ executes the setup algorithm Setup, and produces master secret key *msk* which is kept secretly and parameters *params*, then sends *params* to $\mathcal{A}_2$.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 14,2025 at 03:49:37 UTC from IEEE Xplore. Restrictions apply.

Queries: At this stage, $\mathcal{A}_2$ can conduct the following inquiries of polynomial bounded degree. Furthermore, the following queries are performed adaptively.

1) *Creation-User:* When $\mathcal{A}_2$ asks the creation user query with the identity ID, $\mathcal{C}$ executes ParPriKey to create d, runs SecVal to create x, runs KeyGen to create (PK, SK), and returns PK to $\mathcal{A}_2$.
2) *Extract-Secret-Value*: When $\mathcal{A}_2$ asks this query with ID, $\mathcal{C}$ executes SecVal, and returns x to $\mathcal{A}_2$.
3) *Extract-Private-Key:* When $\mathcal{A}_2$ asks this query with ID, $\mathcal{C}$ executes ParPriKey to create d, runs SecVal to create x, runs KeyGen to create (PK, SK), and returns SK to $\mathcal{A}_2$.
4) *Signature:* When $\mathcal{A}_2$ asks this query with (ID, m), $\mathcal{C}$ executes signature algorithm Sign to generate δ, and returns it to $\mathcal{A}_2$.

Forgery: $\mathcal{A}_2$ outputs a forgery signature δ^* for the challenge identity message pair (ID^*, m^*), when the following conditions are all met, the $\mathcal{A}_2$ wins this game:

1) At any stage, $\mathcal{A}_2$ can not ask *Extract-Private-Key* query for the identity ID^*.
2) $\mathcal{A}_2$ never ask the signature query for (ID^*, m^*), and δ^* is a valid signature.

Therefore, for any PPT adversary $\mathcal{A}_2$, the advantage of winning this game is $\mathsf{Adv}_{\mathcal{A}_2}(\kappa) = \Pr[\mathcal{A}_2\ wins]$.

Definition 2 (Unforgeability): For PPT adversaries $\mathcal{A}_1$ and $\mathcal{A}_2$, if the corresponding advantages $\mathsf{Adv}_{\mathcal{A}_1}(\kappa)$ and $\mathsf{Adv}_{\mathcal{A}_2}(\kappa)$ are negligible in the above games, then, the corresponding CLS scheme has unforgeability.

The nonrepudiation of the CLS scheme requires that the corresponding scheme is undeniable under the chosen message attack, and we define the following game to prove it.

Game 3: This game is executed between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}_3$, the specific execution process is as follows:

Initialization: $\mathcal{C}$ executes the setup algorithm Setup, and produces master secret key msk which is kept secretly and parameters $params$, then sends $params$ to $\mathcal{A}_1$.

Queries: At this stage, $\mathcal{A}_3$ can conduct the following inquiries of polynomial bounded degree. Furthermore, the following queries are performed adaptively.

1) *Creation-User:* When $\mathcal{A}_1$ asks the creation user query with the identity ID, $\mathcal{C}$ executes ParPriKey to create d, runs SecVal to create x, runs KeyGen to create (PK, SK), and returns PK to $\mathcal{A}_3$.
2) *Extract-Partial-Private-Key:* When $\mathcal{A}_3$ asks this query with ID, $\mathcal{C}$ executes ParPriKey and returns d to $\mathcal{A}_3$.
3) *Extract-Secret-Value*: When $\mathcal{A}_3$ asks this query with ID, $\mathcal{C}$ executes SecVal, and returns x to $\mathcal{A}_3$.
4) *Extract-Private-Key:* When $\mathcal{A}_3$ asks this query with the identity ID, $\mathcal{C}$ executes ParPriKey to create d, runs SecVal to create x, runs KeyGen to create (PK, SK), and returns SK to $\mathcal{A}_3$.
5) *Signature:* When $\mathcal{A}_3$ asks this query with the identity message pair (ID, m), $\mathcal{C}$ executes signature algorithm Sign to generate δ, and outputs it to $\mathcal{A}_3$.

Forgery: $\mathcal{A}_3$ outputs a forged public-private key pair (PK^*, SK^*) for the challenge identity ID^*, when the following conditions are all met, the adversary $\mathcal{A}_3$ wins the game:

1) $\mathcal{A}_3$ cannot ask *Extract-Partial-Private-Key* and *Extract-Private-Key* queries for ID^*.
2) The forged public-private key pair (PK^*, SK^*) is valid and different from the pair that is created from creation user query for ID^*.

Therefore, for any PPT adversary $\mathcal{A}_3$, the advantage of winning this game is $\mathsf{Adv}_{\mathcal{A}_3}(\kappa) = \Pr[\mathcal{A}_3\ wins]$.

Definition 3 (Nonrepudiation): For PPT adversary $\mathcal{A}_3$, if the corresponding advantages $\mathsf{Adv}_{\mathcal{A}_3}(\kappa)$ is negligible in the above games, then, the corresponding CLS scheme has nonrepudiation.

IV. Cryptanalysis of Wang et al.'s CLS Scheme

Wang et al.'s scheme [21] includes five algorithms, we review them as follows.

Setup: Input a security parameter, KGC selects a group G with the order q and the generator is P. Computes $P_{\mathrm{pub}} = sP$, sets $s \in Z_q^*$ as system master key. Let $h_1, h_2 : \{0,1\}^* \to Z_q^*$ be two hash functions. Finally, KGC sets public parameters is $params = \{G, P, P_{\mathrm{pub}}, q, h_1, h_2\}$.

Partial Private Key: This algorithm consists of the following operations: The user U_i computes $P_{\mathrm{ID}_i} = \mathrm{ID}_i P$, $C_{\mathrm{ID}_i} = \mathrm{ID}_i + \mathrm{ID}_i P_{\mathrm{pub}}$, where $\mathrm{ID}_i \in \{0,1\}^*$, and sends request message $(P_{\mathrm{ID}_i}, C_{\mathrm{ID}_i})$ to KGC. If $C_{\mathrm{ID}_i} - sP_{\mathrm{ID}_i} = \mathrm{ID}_i$, then KGC selects $r_i \in Z_q^*$, and calculates $R_i = r_i P$, $ppk_i = r_i + s\alpha_i \bmod q$, where $\alpha_i = h_1(P_{\mathrm{pub}}, \mathrm{ID}_i, R_i)$. Next, KGC returns ppk_i to user U_i.

Public/Private Key: The user U_i chooses $x_i \in Z_q^*$ as secret value, and calculates $X_i = x_i P$. Finally, set public key as $PK_i = (X_i, R_i)$ and private key as $SK_i = (x_i, ppk_i)$.

Signature: For the message $m_i \in \{0,1\}^*$, the user U_i computes $\beta_i = h_2(P_{\mathrm{pub}}, \mathrm{ID}_i, PK_i, m_i)$ and $sig_i = (ppk_i - \beta_i x_i) \bmod q$. Finally, sends $\delta_i = (PK_i, sig_i)$ to the verifier V_j.

Verification: For $\beta_i = h_2(P_{\mathrm{pub}}, \mathrm{ID}_i, PK_i, m_i)$ and $\alpha_i = h_1(P_{\mathrm{pub}}, \mathrm{ID}_i, R_i)$, V_j will verify if $sig_i P = R_i + \alpha_i P_{\mathrm{pub}} - \beta_i X_i$ holds.

Notice that, through our analysis, we found that the above CLS scheme has the following shortcomings.

1) Wang et al. set $\mathrm{ID}_i \in \{0,1\}^*$, but there is no definition for the operation of multiplying the string by the generator in the addition group, so it is wrong. That is, $\mathrm{ID}_i P$ and $\mathrm{ID}_i P_{\mathrm{pub}}$ are incalculable.
2) We find that Wang et al.'s CLS scheme is vulnerable for the Type-II adversary. In the signature generation algorithm, U_i calculates $sig_i = (ppk_i - \beta_i x_i) \bmod q$. Because the Type II adversary is a curious and honest KGC, it can get the partial private key ppk_i. Furthermore, sig_i and β are public. In this case, x_i can be obtained by calculating $x_i = ([ppk_i - sig_i]/\beta_i)$. Therefore, the Type-II adversary can obtain the private key SK_i and will forge signatures at will.
3) The signature operation of this CLS scheme does not use a random number, which is not a randomized signature. Therefore, a Type II adversary can forge a valid signature using the existing signature. The scheme is insecure, and it is not suitable for real-world application.

Based on the above analysis, the construction of a secure CLS scheme necessitates rigorous attention to the following

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 14,2025 at 03:49:37 UTC from IEEE Xplore. Restrictions apply.

critical aspects: For the malicious user, it is imperative to prevent such adversaries from obtaining the system master key and to neutralize their capability to compromise the role of the master key in signature validity verification through substitution operations. For the KGC, we should block these adversaries from acquiring user-specific secret information to safeguard individual privacy credentials against unauthorized extraction or abuse.

V. Our Novel Construction of CLS Scheme

Our improved CLS scheme with better security is created in this section.

A. Concrete Construction

The five algorithms of our CLS scheme are illustrated as follows.

1) *Setup:* KGC inputs a security parameter κ, chooses an additive group G with order q and the generator is P. Chooses $s \in Z_q^*$ as the system master secret key and compute $P_{\text{pub}} = sP$. Defines four one-way hash functions $H_0 : G \times \{0,1\}^* \rightarrow \{0,1\}^*$, $H_1, H_2, H_3, H_4 : \{0,1\}^* \rightarrow Z_q^*$. For convenience, we define the corresponding input for the above four hash functions as strings with arbitrarily long. Finally, set $Params = \{G, q, P, P_{\text{pub}}, H_0, H_1, H_2, H_3, H_4\}$ as system public parameters.
2) *Partial Private Key Generation:* The user U_i chooses $x \in Z_q^*$ as secret value and then calculates $X = xP$. Afterward, U_i sends the real identity RID and the public parameter X to KGC. After receiving RID and X from U_i, KGC selects $r \in Z_q^*$, then computes the pseudonym $\text{ID} = RID \oplus H_0(rP_{\text{pub}}, V_t)$, where V_t is the validity period of ID. Next, KGC calculates $d = (r + sh_1) \bmod q$ and $R = rP$, where $h_1 = H_1(P_{\text{pub}}, \text{ID}, R, X)$. Finally, returns (d, R) as the partial key. Notice that, in our construct, the real identity is replaced by a temporary identity generated by KGC, providing anonymity for the user. In addition, anonymity is controllable, as KGC can reveal the malicious real identity.
3) *Key Generation:* User U_i verifies the effectiveness of pseudonym ID and the correctness of partial key (d, R) by calculating $dP = R + h_1 P_{\text{pub}}$. Afterward, U_i sets $SK = (x, d)$ as the the private key, and $PK = (X, R)$ as the public key.
4) *Signature:* Let Time be a time stamp. For the message $m \in \{0,1\}^*$, the signer U_i randomly selects $t \in Z_q^*$, compute $\sigma = h_4 t + h_3 d + h_2 x$ and $T = tP$ where $h_2 = H_2(P_{\text{pub}}, \text{ID}, PK, m\|Time, T)$, $h_3 = H_3(P_{\text{pub}}, \text{ID}, PK, m\|Time, T)$ and $h_4 = H_4(P_{\text{pub}}, \text{ID}, PK, m\|Time, T)$. Finally, U_i outputs $\delta = (T, \sigma)$ as signature and uploads it to cloud server.
5) *Verify:* Verfier obtains δ from cloud server. First check times tamp Time, if it is valid, then continue the verification process. For the signature $\delta = (T, \sigma)$ and the message m, the verfier compute $h_1 = H_1(P_{\text{pub}}, \text{ID}, R, X)$, $h_2 = H_2(P_{\text{pub}}, \text{ID}, m\|Time, PK, T)$, $h_3 = H_3(P_{\text{pub}}, \text{ID}, m\|Time, PK, T)$, and $h_4 = H_4(P_{\text{pub}}, \text{ID}, PK, m\|Time, T)$, if $\sigma P = h_4 T + h_3(R + h_1 P_{\text{pub}}) + h_2 X$ holds, then outputs "1"; Otherwise, verifier outputs "0."

B. Correctness

The correctness of our proposal can be achieved from the equations as follows:

$$\begin{aligned} dP &= (r + sh_1)P = R + h_1 P_{\text{pub}} \\ \sigma P &= (h_4 t + h_3 d + h_2 x)P \\ &= h_4 T + h_3\big(R + h_1 P_{\text{pub}}\big) + h_2 X. \end{aligned}$$

C. Security Proof

In this part, we prove our scheme has better security by demonstrating it can resist the attack of Type-I adversary $\mathcal{A}_1$, Type-II adversary $\mathcal{A}_2$, and Type-III adversary $\mathcal{A}_3$, the detailed poof process is the following.

Theorem 1: If there exists a Type-I adversary $\mathcal{A}_1$ who can break the unforgeability of the CLS scheme with obvious advantage ε_1, then there exists a challenger $\mathcal{C}$ who can solve the hardness of DL assumption with nonnegligible advantage $(1 - [1/e])(\varepsilon_1/[e(Q_1 + Q_2 + 1)Q_h])$. We denote the queries times of H_1 oracle, partial private key generation and private key generation as Q_h, Q_1, Q_2, and e is the natural log base.

Proof: Let $\mathcal{A}_1$ be a Type-I adversary who can forge the signature with obvious advantage ε_1. Now, we construct a challenger $\mathcal{C}$ who can solve the hardness of DL assumption by using $\mathcal{A}_1$. In the beginning, a challenge tuple $(P, Q = sP)$ of DL assumption should be obtained by $\mathcal{C}$, and the goal is to find $s \in Z_q^*$. The game between $\mathcal{C}$ and $\mathcal{A}_1$ is the following.

Initialize: $\mathcal{C}$ sets $P_{\text{pub}} = Q = sP$, and initializes five lists L_1, L_2, L_3, L_4, and L_u used to record the queried message. Finally, $\mathcal{C}$ returns $Params = \{G, q, P, P_{\text{pub}}, H_0, H_1, H_2, H_3, H_4\}$ to $\mathcal{A}_1$, where H_1, H_2, and H_3 are three random oracles.

Queries: $\mathcal{A}_1$ adaptively asks the queries as follows at this stage. The challenger $\mathcal{C}$ chooses a random identity ID^* from the space submitted by $\mathcal{A}_1$ as the challenge identity. Specially, from here on, the parameters related to the challenging identity ID^* are marked with "$*$."

1) *Creation User:* When $\mathcal{A}_1$ submits the query of creation user with identity ID as input, if $(\text{ID}, X, R, x, d) \in L_u$, $\mathcal{C}$ outputs the public key $PK = (X, R)$ to $\mathcal{A}_1$; Otherwise, $\mathcal{C}$ selects random values $d, x, h_1 \in Z_q^*$, calculates $R = dP - h_1 P_{\text{pub}}$ and $X = xP$, then adds (ID, R, h_1) into the list L_1 and (ID, X, R, x, d) into the list L_u. Finally, $\mathcal{C}$ returns $PK = (X, R)$ to $\mathcal{A}_1$. Notice that, for d, there exists a random value $r \in Z_q^*$ that satisfies $d = r + sh_1$.
2) *Partial Private Key:* When $\mathcal{A}_1$ submits the partial private key generation query with inputting ID, $\mathcal{C}$ will abort if $\text{ID} = \text{ID}^*$; Otherwise, $\mathcal{C}$ uses the ID as index to seek a tuple (ID, X, R, x, d) from L_u, if it exists, $\mathcal{C}$ returns (d, R) to $\mathcal{A}_1$; else, $\mathcal{C}$ runs the creation user query for ID and outputs (d, R) to $\mathcal{A}_1$.
3) *Secret Value:* When $\mathcal{A}_1$ submits a secret value generation query with ID as input, if $\text{ID} = \text{ID}^*$, then $\mathcal{C}$ will abort; Otherwise, $\mathcal{C}$ seeks a tuple (ID, X, R, x, d) from L_u with

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 14,2025 at 03:49:37 UTC from IEEE Xplore. Restrictions apply.

ID as index, if it exists, then $\mathcal{C}$ returns x to $\mathcal{A}_1$; else, $\mathcal{C}$ executes the query of creation user for ID and outputs x to $\mathcal{A}_1$.

4) *Private Key:* When $\mathcal{A}_1$ submits the query of private key generation with inputting ID, if $\mathrm{ID} = \mathrm{ID}^*$, $\mathcal{C}$ will abort; Otherwise, $\mathcal{C}$ make ID as an index to seeks the tuple $(\mathrm{ID}, X, R, x, d)$ from L_u, if it exists, $\mathcal{C}$ returns $SK = (x, d)$ to $\mathcal{A}_1$; else, $\mathcal{C}$ executes the creation user query for ID and returns $SK = (x, d)$ to $\mathcal{A}_1$.
5) *H_1 Query:* When $\mathcal{A}_1$ submits an H_1 oracle query with $(\mathrm{ID}, R, P_{\mathrm{pub}})$ as input, $\mathcal{C}$ returns h_1 to $\mathcal{A}_1$ from the list L_1 with $(\mathrm{ID}, R, P_{\mathrm{pub}})$ as an index.
6) *H_2 Query:* When $\mathcal{A}_1$ submits an H_2 oracle query with the tuple $(\mathrm{ID}, PK, P_{\mathrm{pub}}, m, T)$ as input, if $(\mathrm{ID}, PK, P_{\mathrm{pub}}, m, T, h_2) \in L_2$, then $\mathcal{C}$ outputs h_2 to $\mathcal{A}_1$; Otherwise $\mathcal{C}$ chooses a random value $h_2 \in Z_q^*$, adds $(\mathrm{ID}, PK, P_{\mathrm{pub}}, m, T, h_2)$ into the list L_2 and sends it to $\mathcal{A}_1$.
7) *H_3 Query:* When $\mathcal{A}_1$ submits an H_3 oracle query with the tuple $(\mathrm{ID}, PK, P_{\mathrm{pub}}, m, T)$ as input, if $(\mathrm{ID}, PK, P_{\mathrm{pub}}, m, T, h_3) \in L_3$, then $\mathcal{C}$ outputs h_3 to $\mathcal{A}_1$; Otherwise $\mathcal{C}$ chooses a random value $h_3 \in Z_q^*$, adds $(\mathrm{ID}, PK, P_{\mathrm{pub}}, m, T, h_3)$ into the list L_3 and sends it to $\mathcal{A}_1$.
8) *H_4 Query:* When $\mathcal{A}_1$ submits an H_4 oracle query with the tuple $(\mathrm{ID}, PK, P_{\mathrm{pub}}, m, T)$ as input, if $(\mathrm{ID}, PK, P_{\mathrm{pub}}, m, T, h_4) \in L_4$, then $\mathcal{C}$ outputs h_4 to $\mathcal{A}_1$; Otherwise $\mathcal{C}$ chooses a random value $h_4 \in Z_q^*$, adds $(\mathrm{ID}, PK, P_{\mathrm{pub}}, m, T, h_4)$ into the list L_4 and sends it to $\mathcal{A}_1$.
9) *Public Key Replacement:* When $\mathcal{A}_1$ submits a public key replacement query with $(\mathrm{ID}, PK' = (X', R'))$ as input, $\mathcal{C}$ will ignore this query if $\mathrm{ID} = \mathrm{ID}^*$ holds; Otherwise, $\mathcal{C}$ seeks whether $(\mathrm{ID}, X, R, x, d)$ is present in the list L_u, if it exists, $\mathcal{C}$ updates it with tuple $(\mathrm{ID}, X', R', \perp, \perp)$, else, $\mathcal{C}$ executes the query of creation user query with ID and then replaces $(\mathrm{ID}, X, R, x, d)$ with $(\mathrm{ID}, X', R', \perp, \perp)$.
10) *Signature:* When $\mathcal{A}_1$ submits a signature generation query with inputting the identity message pair (ID, m_i), $\mathcal{C}$ performs the following operations.

1) If $\mathrm{ID} \neq \mathrm{ID}^*$ and $d \neq \perp$, $\mathcal{C}$ chooses $t \in Z_q^*$, then calculates $T = tP$, $h_2 = H_2(P_{\mathrm{pub}}, \mathrm{ID}, m, PK)$, $h_3 = H_3(P_{\mathrm{pub}}, \mathrm{ID}, m, PK, T)$ and $h_4 = H_4(P_{\mathrm{pub}}, \mathrm{ID}, m, PK, T)$. Next $\mathcal{C}$ calculates $\sigma = h_4 t + h_3 d + h_2 x$ and outputs $\delta = (T, \sigma)$ to $\mathcal{A}_1$.

2) Otherwise, $\mathcal{C}$ selects $\sigma, h_2, h_3, h_4 \in Z_q^*$, and calculates $T = (1/h_4)(\sigma P - h_3(R + h_1 P_{\mathrm{pub}}) - h_2 X)$. Afater that, adds $(\mathrm{ID}, PK, P_{\mathrm{pub}}, m_i, T, h_2)$, $(\mathrm{ID}, PK, P_{\mathrm{pub}}, m_i, T, h_3)$ and $(\mathrm{ID}, PK, P_{\mathrm{pub}}, m_i, T, h_4)$ into the lists L_2, L_3 and L_4. Finally, $\mathcal{C}$ outputs the signature $\delta = (T, \sigma)$ to $\mathcal{A}_1$. Specifically, h_1, X and R can be obtained from the corresponding lists with ID as an index.

Forgery: Finally, $\mathcal{A}_1$ returns a forged signature $\delta_1^* = (T^*, \delta_1^*)$ for the challenge identity message pair (ID_i, m_i^*), where $T^* = t^* P$. If $\mathrm{ID}_i \neq \mathrm{ID}^*$, $\mathcal{C}$ aborts. Otherwise, based on Forking lemma [30], $\mathcal{A}_1$ can forge a new signature $\delta_2^* = (T^*, \sigma_2^*)$ by the different hash response $\hat{h}_1^*$ of H_1 oracle and same random value t^*, which makes these two equations $\sigma_1^* = h_4^* t^* + h_3^*(r^* + s h_1^*) + h_2^* d^*$ and $\sigma_2^* = h_4^* t^* + h_3^*(r^* + s \hat{h}_1^*) + h_2^* d^*$ hold.

The result shows $s = ([\sigma_1^* - \sigma_2^*]/[h_3^*(h_1^* - \hat{h}_1^*)])$, which is against the DL assumption. ξ_1 indicates that at the query stage the above game does not abort; ξ_2 indicates that at the forgery stage the above game does not abort; ξ_3 indicates $\mathcal{A}_1$ creating two valid signatures δ_1^* and δ_2^*. Therefore, we get $\Pr[\xi_1] = [1 - (1/[Q_1 + Q_2 + 1])]^{Q_1 + Q_2}$ and $\Pr[\xi_2] = (1/[Q_1 + Q_2 + 1])$. Through Forking lemma, the probability of $\mathcal{A}_1$ creating two valid forged signatures δ_1^* and δ_2^* is $\Pr[\xi_3] \geq (1 - [1/e])(\varepsilon_1/Q_h)$. Then, we have

$$\begin{aligned}
&\Pr[\xi_1 \wedge \xi_2 \wedge \xi_3] \\
&\geq \left(1 - \frac{1}{Q_1 + Q_2 + 1}\right)^{Q_1 + Q_2} \frac{1}{Q_1 + Q_2 + 1}\left(1 - \frac{1}{e}\right)\frac{\varepsilon_1}{Q_h} \\
&\geq \left(1 - \frac{1}{e}\right)\frac{\varepsilon_1}{e(Q_1 + Q_2 + 1)Q_h}.
\end{aligned}$$

In conclusion, if there exists an adversary $\mathcal{A}_1$ who can break the unforgeability of the CLS scheme with advantage ε_1, then there exists a challenger $\mathcal{C}$ who can solve the hardness of DL assumption with nonnegligible probability $(1 - [1/e])(\varepsilon_1/[e(Q_1 + Q_2 + 1)Q_h])$. ■

Theorem 2: If there exists the Type-II adversary $\mathcal{A}_2$ who can can break the unforgeability of our CLS scheme with obvious advantage ε_2, then there exists a challenger $\mathcal{C}$ who can solve the hardness of DL assumption with non-negligible advantage $(1 - [1/e])(\varepsilon_2/[e(Q_2 + Q_3 + 1)Q_h'])$, where Q_3 and Q_h' are the number of secret value generation and H_2 oracle queries.

Proof: Let $\mathcal{A}_2$ be a Type–II adversary who can forge the valid signature with obvious advantage ε_2. Now, we construct a challenger $\mathcal{C}$ who can solve the hardness of DL assumption by using $\mathcal{A}_2$. In the beginning, a challenge tuple $(P, Q = aP)$ of DL assumption should be obtained by $\mathcal{C}$, the goal is to find $a \in Z_q^*$. The game between $\mathcal{C}$ and $\mathcal{A}_1$ is the following:

Initialize: The challenger $\mathcal{C}$ chooses $s \in Z_q^*$, calculates $P_{\mathrm{pub}} = sP$, and initializes four lists L_2, L_3, L_4, and L_u used to record the queried message. Finally, $\mathcal{C}$ returns $Params = \{G, q, P, P_{\mathrm{pub}}, H_0, H_1, H_2, H_3, H_4\}$ to $\mathcal{A}_1$, where H_2, H_3, H_4 are three random oracles.

Queries: $\mathcal{A}_2$ adaptively asks the queries as follows at this stage. The challenger $\mathcal{C}$ chooses a random identity ID^* from the space submitted by $\mathcal{A}_2$ as the challenge identity. Furthermore, $\mathcal{C}$ responds to H_2, H_3, and H_4 queries in the same way as Theorem 1.

1) *Creation User:* When $\mathcal{A}_2$ submits a query of creation user with inputting ID, if $(\mathrm{ID}, X, R, x, d) \in L_u$, then $\mathcal{C}$ returns public key $PK = (X, R)$ to $\mathcal{A}_2$; Otherwise, $\mathcal{C}$ performs the following operations.
   a) If $\mathrm{ID} = \mathrm{ID}^*$, then $\mathcal{C}$ makes $X^* = aP$, selects random values $r^* \in Z_q^*$, calculates $R^* = r^* P$ and $d^* = r^* + s h_1^*$, where $h_1^* = H_1(P_{\mathrm{pub}}, \mathrm{ID}^*, R^*)$, then adds $(\mathrm{ID}^*, X^*, R^*, \perp, d^*)$ into L_u. Finally, $\mathcal{C}$ returns $PK^* = (X^*, R^*)$ to $\mathcal{A}_2$.
   b) If $\mathrm{ID} \neq \mathrm{ID}^*$, then $\mathcal{C}$ selects $x, r \in Z_q^*$, calculates $X = xP$, $R = rP$ and $d = r + s h_1$, where $h_1 =$

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 14,2025 at 03:49:37 UTC from IEEE Xplore. Restrictions apply.

$H_1(P_{\text{pub}}, \text{ID}, R)$. $\mathcal{C}$ inserts the tuple (ID, X, R, x, d) in the list L_u. Finally, $\mathcal{C}$ outputs $PK = (X, R)$ to $\mathcal{A}_2$.

2) *Secret Value:* When $\mathcal{A}_2$ submits the query of secret value generation with inputting ID, if $\text{ID} = \text{ID}^*$, then $\mathcal{C}$ aborts; Otherwise, $\mathcal{C}$ seeks a tuple (ID, X, R, x, d) from L_u with ID as an index, if it exists, then $\mathcal{C}$ returns x; Otherwise, $\mathcal{C}$ executes the creation user query for ID and returns x.
3) *Private Key:* When $\mathcal{A}_2$ submits a private key generation query with ID as input, if $\text{ID} = \text{ID}^*$, then $\mathcal{C}$ aborts; Otherwise, $\mathcal{C}$ make ID as index to seek a tuple (ID, X, R, x, d) from L_u, if it exists, $\mathcal{C}$ outputs $SK = (x, d)$ to $\mathcal{A}_2$; else, $\mathcal{C}$ will execute the query of creation user for ID and returns $SK = (x, d)$ to $\mathcal{A}_2$.
4) *Signature:* When $\mathcal{A}_2$ submits the signature generation query with inputting (ID, m_i), $\mathcal{C}$ selects $\sigma, h_2, h_3, h_4 \in Z_q^*$, and calculates $T = (1/h_4)(\sigma P - h_3(R + h_1 P_{\text{pub}}) - h_2 X)$, where $h_1 = H_1(P_{\text{pub}}, \text{ID}, R)$, R and X can be obtain by performing creation user query for ID. Also, adds tuples $(\text{ID}, PK, P_{\text{pub}}, m_i, T, h_2)$, $(\text{ID}, PK, P_{\text{pub}}, m_i, T, h_3)$ and $(\text{ID}, PK, P_{\text{pub}}, m_i, T, h_4)$ into the lists L_2,L_3 and L_4. Finally, $\mathcal{C}$ returns $\delta = (T, \sigma)$ as the signature to $\mathcal{A}_2$.

Forgery: Finally, $\mathcal{A}_2$ forges a signature $\delta_1^* = (T^*, \sigma_1^*)$ for the challenge identity message pair (ID_i, m_i^*). If $\text{ID}_i \neq \text{ID}^*$, then $\mathcal{C}$ aborts, and output $\perp$. Otherwise, based on Forking Lemma [30], $\mathcal{A}_2$ produces $\delta_2^* = (T^*, \sigma_2^*)$ as a new forged signature with different hash response $\hat{h}_2$ of H_2 oracle and the same random value t^*, which makes these two equations $\sigma_1^* = h_4^* t + h_3^*(r^* + s h_1^*) + h_2^* a$ and $\sigma_2^* = h_4^* t + h_3^*(r^* + s h_1^*) + \hat{h}_2^* a$ hold.

The result shows $a = ([\sigma_1^* - \sigma_2^*]/[h_2^* - \hat{h}_2^*])$, which is against the DL assumption. $\mathcal{F}_1$ indicates that at the query stage the above game does not abort; $\mathcal{F}_2$ indicates that at the forgery stage the above game does not abort; $\mathcal{F}_3$ indicates $\mathcal{A}_2$ creating two valid signatures δ_1^* and δ_2^*. Therefore, we get $\Pr[F_1] = (1 - (1/[Q_2 + Q_3 + 1]))^{Q_2 + Q_3}$ and $\Pr[F_2] = (1/[Q_2 + Q_3 + 1])$. Through Forking lemma, the probability of $\mathcal{A}_2$ creating two valid forge signatures δ_1^* and δ_2^* is $\Pr[F_3] \geq (1 - [1/e])(\varepsilon_2/Q_h')$. Then, we have $\Pr[F_1 \wedge F_2 \wedge F_3] \geq (1 - [1/e])(\varepsilon_1/[e(Q_2 + Q_3 + 1)Q_h'])$.

In conclusion, if there exists an adversary $\mathcal{A}_2$ who can break the unforgeability of our CLS scheme with advantage ε_2, then there exists a challenger $\mathcal{C}$ who can solve the hardness of DL assumption with non-negligible probability $(1 - [1/e])(\varepsilon_2/[e(Q_2 + Q_3 + 1)Q_h'])$. ■

Theorem 3: If there exists a Type-III adversary $\mathcal{A}_3$ who can break the Nonrepudiation of the CLS scheme with obvious advantage ε_3, then there exists a challenger $\mathcal{C}$ who can break the ElGamal signature scheme with non-negligible advantage $(\varepsilon_3/[e(Q_1 + Q_2 + 1)Q_h])$. We denote the queries times of H_1 oracle, partial private key generation and private key generation as Q_h, Q_1, Q_2, and e is the natural log base.

Proof: Let $\mathcal{A}_3$ be a Type-III adversary who can forge the public-private key pair with obvious advantage ε_3. Now, we construct a challenger $\mathcal{C}$ who can break the ElGamal signature scheme by using $\mathcal{A}_3$. The game between $\mathcal{C}$ and $\mathcal{A}_3$ is the following.

Initialize: $\mathcal{C}$ randomly selects $a, h \in Z_q^*$ and sets $P_{\text{pub}} = sP'$ where P' is the public key of ElGamal signature scheme and $s \in Z_q^*$ is the master secret key. $\mathcal{C}$ initializes four lists L_2, L_3, L_4, and L_u used to record the queried message. Finally, $\mathcal{C}$ returns $Params = \{G, q, P, P_{\text{pub}}, H_0, H_1, H_2, H_3\}$ to $\mathcal{A}_1$, where H_1, H_2 and H_3 are three random oracles.

Queries: $\mathcal{A}_3$ adaptively asks the queries as follows at this stage. The challenger $\mathcal{C}$ responds to H_2, H_3, and H_4 queries in the same way as Theorem 1. Furthermore, $\mathcal{C}$ chooses a random identity ID^* from the space submitted by $\mathcal{A}_3$ as the challenge identity.

1) *Creation User:* When $\mathcal{A}_3$ submits the query of creation user with identity ID as input, if $(\text{ID}, X, R, x, d) \in L_u$, $\mathcal{C}$ outputs the public key $PK = (X, R)$ to $\mathcal{A}_3$; Otherwise, $\mathcal{C}$ selects random values $r, x \in Z_q^*$, calculates $R = rP$, $X = xP$ and $d = r + sH_1(\text{ID}, X, R, P_{\text{pub}})$, then adds (ID, X, R, x, d) into the list L_u. Finally, $\mathcal{C}$ returns $PK = (X, R)$ to $\mathcal{A}_3$.
2) *Partial Private Key:* When $\mathcal{A}_3$ submits the partial private key generation query with inputting ID, $\mathcal{C}$ will abort if $\text{ID} = \text{ID}^*$; Otherwise, $\mathcal{C}$ uses the ID as index to seek a tuple (ID, X, R, x, d) from L_u, if it exists, $\mathcal{C}$ returns (d, R) to $\mathcal{A}_3$; else, $\mathcal{C}$ runs the creation user query for ID and outputs (d, R) to $\mathcal{A}_3$.
3) *Secret Value:* When $\mathcal{A}_3$ submits a secret value generation query with ID as input, if $\text{ID} = \text{ID}^*$, then $\mathcal{C}$ aborts; Otherwise, $\mathcal{C}$ seeks (ID, X, R, x, d) from L_u with ID as index, if it exists, then $\mathcal{C}$ returns x to $\mathcal{A}_3$; else, $\mathcal{C}$ will execute the query of creation user for ID and outputs x to $\mathcal{A}_3$.
4) *Private Key:* When $\mathcal{A}_3$ submits the query of private key generation with inputting ID, if $\text{ID} = \text{ID}^*$, $\mathcal{C}$ aborts; Otherwise, $\mathcal{C}$ make ID as index to seeks the tuple (ID, X, R, x, d) from L_u, if it exists, $\mathcal{C}$ returns $SK = (x, d)$ to $\mathcal{A}_3$; else, $\mathcal{C}$ executes the creation user query for ID and returns $SK = (x, d)$ to $\mathcal{A}_3$.
5) *Signature:* When $\mathcal{A}_3$ submits a signature generation query with inputting (ID, m_i), $\mathcal{C}$ performs the following operations.

1) If $\text{ID} \neq \text{ID}^*$, $\mathcal{C}$ chooses $t \in Z_q^*$, then calculates $T = tP$, $h_2 = H_2(P_{\text{pub}}, \text{ID}, m, PK)$, $h_3 = H_3(P_{\text{pub}}, \text{ID}, m, PK, T)$ and $h_4 = H_4(P_{\text{pub}}, \text{ID}, m, PK, T)$. Next $\mathcal{C}$ calculates $\sigma = h_4 t + h_3 d + h_2 x$ and outputs $\delta = (T, \sigma)$ as signature to $\mathcal{A}_3$.

2) Otherwise, $\mathcal{C}$ selects $\sigma, h_2, h_3, h_4 \in Z_q^*$, and calculates $T = (1/h_4)(\sigma P - h_3(R + h_1 P_{\text{pub}}) - h_2 X)$, where $h_1 = H_1(\text{ID}, X, R, P_{\text{pub}})$. After that, adds $(\text{ID}, PK, P_{\text{pub}}, m_i, T, h_2)$, $(\text{ID}, PK, P_{\text{pub}}, m_i, T, h_3)$ and $(\text{ID}, PK, P_{\text{pub}}, m_i, T, h_4)$ into the lists L_2, L_3 and L_4. Finally, $\mathcal{C}$ outputs the signature $\delta = (T, \sigma)$ to $\mathcal{A}_3$.

Forgery: Finally, $\mathcal{A}_3$ returns a forged public-private key pair (PK^*, SK^*) for the challenge identity ID_i, where $SK^* = (x^*, d^*) = (x^*, r^* + sH_1(\text{ID}, R^*, P_{\text{pub}}))$ and $PK^* = (X^*, R^*) = (x^*P, r^*P)$. If $\text{ID}_i \neq \text{ID}^*$, then $\mathcal{C}$ aborts; Otherwise, $\mathcal{C}$ outputs (R^*, d^*) as a forged signature of ElGamal scheme on the challenge message $(\text{ID}^*, X^*, P_{\text{pub}})$.

Notice that, if (PK^*, SK^*) is a valid public-private key pair, then (R^*, d^*) is a valid forged signature of the ElGamal

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 14,2025 at 03:49:37 UTC from IEEE Xplore. Restrictions apply.

TABLE II
RUNNING COST OF BASE CRYPTOGRAPHY OPERATIONS

Symbol	Algorithm	Time Required (ms)
T_{pa}	A point addition on elliptic curve	0.0021
T_{ps}	A point subtraction on elliptic curve	0.0021
T_{sm}	A scalar multiplication on elliptic curve	0.4412
T_{me}	A modular exponentiation operation	3.7043
T_{rc}	A modular inversion computation	0.1920
T_{bp}	Bilinear Pairing operation	2.5521

scheme on the challenge message $(\mathrm{ID}^*, X^*, P_{\mathrm{pub}})$, because the equation $d^*P' = R^* + H_1(\mathrm{ID}^*, X^*, R^*, P_{\mathrm{pub}})P_{\mathrm{pub}}$ holds.

We use $\mathcal{N}_1$ to indicate that at the query stage the above game does not abort; $\mathcal{N}_2$ indicates that at the forgery stage the above game does not abort; $\mathcal{N}_3$ indicates $\mathcal{A}_3$ creating valid key pair (PK^*, SK^*), that means (PK^*, SK^*) is the valid signature of ElGamal signature scheme for the message $(\mathrm{ID}^*, X^*, P_{\mathrm{pub}})$. Therefore, we get $\Pr[\mathcal{N}_1] = [1 - (1/[Q_1 + Q_2 + 1])]^{Q_1+Q_2}$, $\Pr[\mathcal{N}_2] = (1/[Q_1 + Q_2 + 1])$ and $\Pr[\mathcal{N}_3] = \varepsilon_3$. Then, we have $\Pr[\mathcal{N}_1 \wedge \mathcal{N}_2 \wedge \mathcal{N}_3] \geq (\varepsilon_3/[e(Q_1 + Q_2 + 1)Q_h])$.

In conclusion, if there exists an adversary $\mathcal{A}_3$ who can break the non-repudiation of our CLS scheme with advantage ε_3, then there exists a challenger $\mathcal{C}$ who can break the ElGamal signature scheme with non-negligible probability $(\varepsilon_3/[e(Q_1 + Q_2 + 1)Q_h])$. ■

D. Performance Analysis

In this section, we will compare our CLS scheme with other CLS schemes integrally from the aspects of computation and communication. We use the Charm-crypto library to perform the simulation experiment based on pycharm, where our algorithm runs on Linux, the processor is Intel i7-8550, and the memory is 8G. We show the the base operations' cost in Table II.

1) Communication Cost Comparison: The signature length is a prominent element in deciding the communication cost. We use $|G|$ to express the size of an additional group $|G|$, $|G_1|$ to denote the size of a multiplicative group $|G_1|$, $|Z_q^*|$ to express the group element's size in the finite field. Hence, our communication cost is $|G| + |Z_q^*|$ (480 bits). But through our previous security analysis, the corresponding constructions [15], [16], [21], [27] are vulnerable to the Type-I adversaries' attack, and scheme [21] can not resist the Type-II adversary. Furthermore, the length of signature [18], [26], [28] is too long, so they have high communication costs. Therefore, our scheme achieves lower communication costs and better security. We use Table III to describe the comparison results of communication costs.

2) Computation Cost Comparison: In this part, we mainly consider the cost of the signature and verification algorithms. From Tables II, IV, and Fig. 1, we can find that the computation cost of our scheme is $5T_{sm} + 2T_{pa} + T_{ps} = 2.0623$(ms). Based on Table II, we can obtain the proposals [18], [26], [28] contain bilinear mapping operation, and they have the highest calculation costs. Although the computation efficiency of constructions is low, they can not resist the signature forgery attack [15], [16], [19], [21], [27]. Furthermore, the existing constructs do not have anonymity [15], [16], [17], [19].

TABLE III
COMPARISON OF COMMUNICATION COST

Scheme	Signature length	Communication cost
Scheme [15]	$\|G\|+\|Z_q^*\|$	480bits
Scheme [16]	$\|G\|+\|Z_q^*\|$	480bits
Scheme [18]	$\|G_1\|+\|Z_q^*\|$	1184bits
Scheme [19]	$\|G\|+\|Z_q^*\|$	480bits
Scheme [21]	$\|G\|+\|Z_q^*\|$	480bits
Scheme [26]	$2\|G_1\|$	2048bits
Scheme [27]	$\|G\|+\|Z_q^*\|$	480bits
Scheme [28]	$2\|G_1\|$	2048bits
our scheme	$\|G\|+\|Z_q^*\|$	480bits

TABLE IV
COMPARISON OF COMPUTATIONAL EFFICIENCY

Scheme	Signature Generation	Verification	Security
scheme [15]	$1T_{sm}$+$1T_{rc}$	$4T_{sm}$+$2T_{pa}$	no
scheme [16]	$1T_{sm}$+$1T_{rc}$	$4T_{sm}$+$2T_{pa}$	no
scheme [18]	$1T_{me}$	$4T_{me}$	yes
scheme [19]	$1T_{sm}$	$4T_{sm}$+$2T_{pa}$	yes
scheme [21]	$2T_{sm}$	$3T_{sm}$+$1T_{pa}$+$1T_{ps}$	no
scheme [26]	$2T_{sm}$	T_{bp}+T_{me}	yes
scheme [27]	$1T_{sm}$	$4T_{sm}$+T_{pa}	no
scheme [28]	$2T_{sm}$	T_{bp}+T_{me}	yes
our scheme	$1T_{sm}$	$4T_{sm}$+$1T_{pa}$+$1T_{ps}$	yes

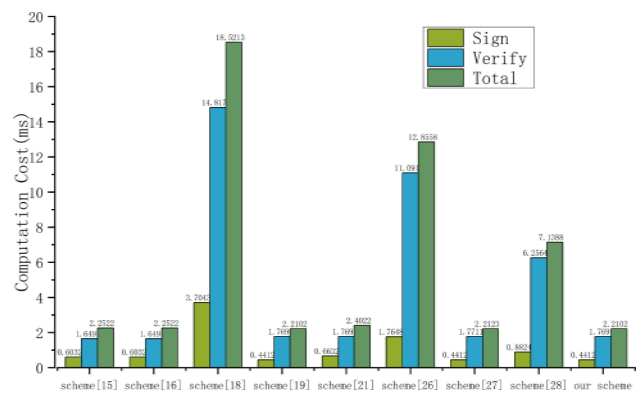

Fig. 1. Comparison results of computation cost.

Therefore, our CLS scheme has high security and computation efficiency and can realize anonymity. Therefore, our scheme is the most suitable for health IIoT with limited resources through comprehensive comparison.

VI. SECURE IDENTITY AUTHENTICATION PROTOCOL FROM OUR CLS SCHEME FOR HEALTHCARE IIOT

In healthcare IIoT, patients and doctors can achieve timely treatment by sharing health data and diagnosis programs. However, to ensure the identity legality of communication entities, patients and doctors should verify each other's identities. Therefore, we propose an identity authentication and key agreement protocol utilizing our CLS scheme that can realize mutual identity authentication between patients and doctors, which can ensure identity legality and data confidentiality. Furthermore, we prove its security and effectiveness through security simulation and performance analysis.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 14,2025 at 03:49:37 UTC from IEEE Xplore. Restrictions apply.

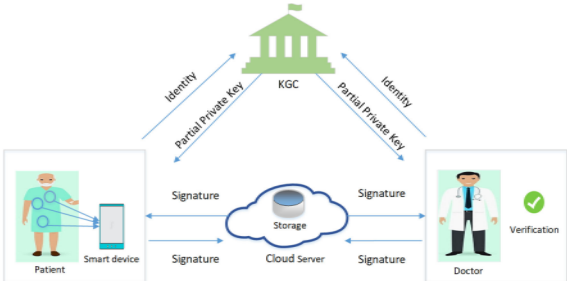

Fig. 2. System framework of health IIoT.

A. System Model

We employ Fig. 2 to show the system framework of health IIoT, and will provide a specific introduction.

The main participating entities are the following.

1) *KGC:* KGC produces public parameters and master private keys for the medical system and generates partial private keys for patients and doctors. In the healthcare system, it plays the role of the hospital management department.
2) *Patient:* Patients' health information (e.g., blood pressure, heart rate, etc.) can be collected by smart devices and uploaded to cloud servers. Finally, patients obtain the diagnostic results from the cloud server uploaded by doctors.
3) *Cloud Sever:* The cloud server can store uploaded body data from patients and uploaded diagnostic results from doctors. The cloud server provides data storage functions for the healthcare IIoT, and provides security protection for data, reducing the pressure of data storage for users. The cloud server facilitates the secure mediation of communication message exchanges, as exemplified in the proposed identity authentication protocol, where patient-physician interactions are orchestrated via cloud server resources to ensure end-to-end communication.
4) *Doctor:* Doctors can download patients' health information from the cloud server, diagnose diseases, and then upload the diagnostic results to the cloud server.

B. Concrete Construction

Initialization: KGC selects an additive group G of order q and the generator is P, and selects $s \in Z_q^*$ to compute $P_{\text{pub}} = sP$. Let $H, H_1, H_2, H_3, H_4, H_5 : \{0,1\}^* \rightarrow Z_q^*$ be six one-way hash functions. Finally, KGC sets $Params$=$\{G, q, p, P_{\text{pub}}, H_1, H_2, H_3, H_4, H_5\}$ as the public parameter.

Registration: The registration operations of the doctor and patient are as follows.

1) Doctor or patient sends the corresponding identity RID to KGC, after receiving the ID, KGC computes the pseudonym $\text{ID} = RID \oplus H_0(rP_{\text{pub}}, V_t)$, where V_t is the validity period of ID. Then, KGC selects $r \in Z_q^*$ to compute $R = rP$ and $d = r + sh_1 \bmod q$, where $h_1 = H_1(P_{\text{pub}}, \text{ID}, R)$, then KGC sets the (d, R) as the partial key.
2) The doctor or patient can verify the partial key's correctness by calculating $dP = R + h_1 P_{\text{pub}}$ and determining the validity of pseudonym ID. Afterwards, doctor or patient sets $x \in Z_q^*$ as the secret value, computes $X = xP$ and sets the private key $SK = (x, d)$ and the public key $PK = (X, R)$.

Identity Authentication: In this part, the doctor and patient achieve mutual identity verification. Define U_a and U_b denote the patient and doctor, Enc and Dec are the algorithms of the symmetric key encryption and symmetric key decryption.

1) The authentication request operations of U_a are as follows. Let Time_a be the current time stamp. Select $t_a, z_a \in Z_q^*$ to calculate $T_a = t_a P$, $Z_a = z_a P$, $W_a = t_a(X_b + R_b + h_1^b P_{\text{pub}})$, and $C_a = \mathsf{Enc}(k_a, T_a || \text{ID}_a || Time_a)$, where $h_1^b = H_1(P_{\text{pub}}, \text{ID}_b, R_b)$ and $k_a = H(W_a)$. Afterward, compute $\sigma_a = h_4^a t_a + h_3^a d_a + h_2^a x_a \bmod q$, where $h_2^a = H_2(P_{\text{pub}}, \text{ID}_a, PK_a, T_a, C_a)$, $h_3^a = H_3(P_{\text{pub}}, \text{ID}_a, PK_a, T_a, C_a)$, and $h_4^a = H_4(P_{\text{pub}}, \text{ID}_a, PK_a, T_a, C_a)$. Finally, U_a sends $(T_a, Z_a, C_a, \sigma_a, Time_a)$ to cloud server.
2) The verification operations of U_b are as follows. Download $(T_a, Z_a, C_a, \sigma_a, Time_a)$ from cloud server. Compute $W_a' = (x_b + d_b)T_a$ and $T_a || \text{ID}_a || Time_a = \mathsf{Dec}(k_a', W_a')$, where $k_a' = H(W_a')$, where Time_a is employed to avoid the replay attacks. Get the decryption algorithm through computing $W_a' = (x_b + d_b)T_a = t_a(x_b + d_b)P = t_a(X_b + R_b + h_1^b P_{\text{pub}}) = W_a$. If $\sigma_a P = h_4^a T_a + h_3^a(R_a + h_1^a P_{\text{pub}}) + h_2^a X_a$, then the validity of U_a is verified.
3) The authentication request operations of U_b are the following. Let Time_b be the current time stamp. select $t_b, z_b \in Z_q^*$ to calculate $T_b = t_b P$, $Z_b = z_b P$, $W_b = t_b(X_a + R_a + h_1^a P\text{pub})$, and $C_b = \mathsf{Enc}(k_b, T_b || \text{ID}_b, Time_b)$, where $h_1^a = H_1(P_{\text{pub}}, id_a, R_a)$ and $k_b = H(W_b)$. Afterward, compute $\sigma_b = h_4^b t_b + h_3^b d_b + h^a b_2 x_b$ and q, where $h_2^b = H_2(P_{\text{pub}}, id_b, PK_b, T_b, C_b)$, $h_3^b = H_3(P_{\text{pub}}, id_b, PK_b, T_b, C_b)$, and $h_4^b = H_4(P_{\text{pub}}, id_b, PK_b, T_b, C_b)$. Finally, U_b sends $(T_b, Z_b, C_b, \sigma_b, Time_b)$ to cloud server.
4) The verification operations of U_a are as follows. Download $(T_b, Z_b, C_b, \sigma_b, Time_b)$ from cloud server. Compute $W_b' = (x_a + d_a)T_b$ and $T_b || \text{ID}_b || Time_b = \mathsf{Dec}(k_b', W_b')$, where $k_b' = H(W_b')$, where Time_b is also employed to avoid the replay attacks.. Get the decryption algorithm through computing $W_b' = (x_a + d_a)T_b = t_b(x_a + d_a)P = t_b(X_a + R_a + h_1^a P_{\text{pub}}) = W_b$. If $\sigma_b P = h_4^b T_b + h_3^b(R_b + h_1^b P_{\text{pub}}) + h_2^b X_b$ holds, then the validity of U_b is verified.

Key Agreement: After verifying the legitimacy of mutual identity, U_a and U_b make the session key K_{ab}.

1) U_a calculates the $K_a = Z_b z_a = z_b z_a P$, then computes the session key $K_{ab} = H_5(U_a, U_b, K_a)$.

2) U_b calculates the $K_b = Z_a z_b = z_a z_b P$, then computes the session key $K_{ab} = H_5(U_a, U_b, K_b)$.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 14,2025 at 03:49:37 UTC from IEEE Xplore. Restrictions apply.

Finally, U_a and U_b utilize the session key K_{ab} to encrypt and decrypt messages.

Communication: After the preceding operations are complete, a mutually authenticated session key is created between the patient and physician. The patient subsequently employs a semantically secure symmetric encryption algorithm to protect personal health information prior to transmission. Following secure delivery of the encrypted data, the physician decrypts the ciphertext using the preshared session key to recover the original information.

C. Security Analysis

1) *Mutual Authentication:* Only legitimate U_a and U_b can generate legitimate request messages. Our CLS scheme can ensure the validity of the message. In the previous introduction, we also proved that our scheme is unforgivable. Therefore, the adversary cannot forge the communication messages of the identity authentication protocol.
2) *Resist Replay Attacks:* In the identity authentication protocol, we use the time stamps Time_a and Time_b used in our protocol to resist replay attacks.
3) *Forward and Backward Security:* Because the value of a random number for creating the session key in each request and response process is different, even if a session key leaks, it will not affect the security of the other. Therefore, our protocol has excellent forward and backward security.
4) *Unlinkability:* When U_a sends a message to U_b, in our scheme, W_a and T_a are two random numbers. Because each request has different values, the adversary does not know the message is sent by the same U_a, so our scheme has unlinkability.
5) *Anonymity:* To ensure the privacy of doctors and patients that protect their identities do not leak, we use pseudonyms instead of their true identities and set a validity period for them. Once the timestamp expires, the pseudonyms will be invalidated. So, our protocol realizes anonymity.
6) *Confidentiality:* After verifying the legitimacy of both parties' identities, the participant generates a session key. In subsequent communication, the user employs the above session key to encrypt the message, ensuring the confidentiality of the data.
7) *Integrity:* The above protocol is created based on our CLS scheme, which ensures that the data has not been modified and protects the integrity of the data through a CLS.
8) *Resist MITM Attacks:* Our protocol employs the underlying CLS scheme to prevent man-in-the-middle attacks, which can ensure the messages are unmodified. Furthermore, we use encrypted messages through session keys to prevent message leakage. Therefore, the malicious attackers can not intercept and modify the communication messages.

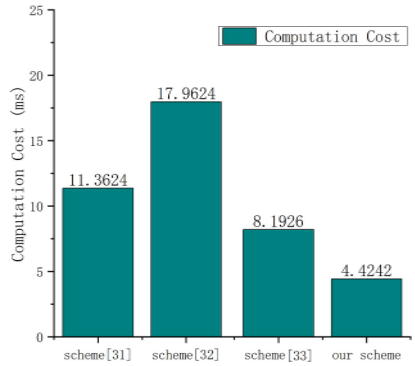

Fig. 3. Comparison results of computation cost of identity authentication.

TABLE V
COMPARISON OF COMPUTATION COST OF PROTOCOL

Scheme	Registration	Authentication	Total
scheme [31]	$4T_{pbsm}$	$2T_{pbsm}$+$3T_{bp}$+$2T_{me}$	$6T_{pbsm}$+$3T_{bp}$+$2T_{me}$
scheme [32]	$2T_{pbsm}$+$3T_{bp}$+T_{me}	T_{pbsm}+T_{bp}+$2T_{me}$	$3T_{pbsm}$+T_{bp}+$3T_{me}$
scheme [33]	$7T_{pbsm}$	$2T_{bp}$	$7T_{pbsm}$+$2T_{bp}$
our scheme	$5T_{sm}$+T_{pa}	$5T_{sm}$+$4T_{pa}$	$10T_{sm}$+$5T_{pa}$

D. Performance Analysis

In this section, we compare the calculation cost of our identity authentication protocol with several identity authentication protocols. We found that Jiang et al.'s protocol [31] is vulnerable to replay attacks. Rasslan et al. [32] designed an identity-based protocol for the medical IoT, which existed as a key-escrow problem. We also found that our mutual identity authentication protocol using a pairing-free CLS scheme has better efficiency, while the calculation cost of the existing protocols [33], based on bilinear mapping is too high.

E. Simulation Analysis

We conducted simulation experiments to apply our scheme to identity authentication and key exchange protocols based on medical systems, in which, if the patient and doctor log in, they send their identities to each other, which can be verified based on fuzzy queries, and they generate a session key to encrypt the transmitted message.

We conducted simulation experiments to apply our protocol to achieve identity authentication and key exchange based on medical systems. After the patient and doctor log in, they send their identities to each other, which can be verified based on fuzzy queries. Finally, we generate a session key used to encrypt the transmitted message. The above simulation experiment runs on a Windows operation system, with the backend using the Spring+SpringMVC+MyBatis (SSM) framework and the front end using the View framework and Element components, and we use the MySql database to store user identity information. Fig. 4 is the user interface, where patients and doctors enter interfaces through their respective accounts. The patient can query the identity sent by the doctor and send their ID to the doctor for verification. The doctor conducts a fuzzy query to find the ID sent by the patient, and we use Fig. 5 to describe the above operations.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 14,2025 at 03:49:37 UTC from IEEE Xplore. Restrictions apply.

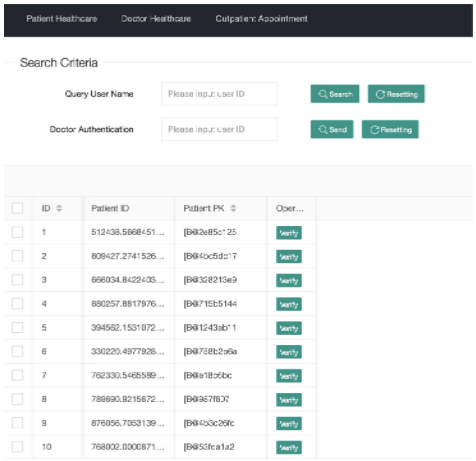

Fig. 4. User interface of our system.

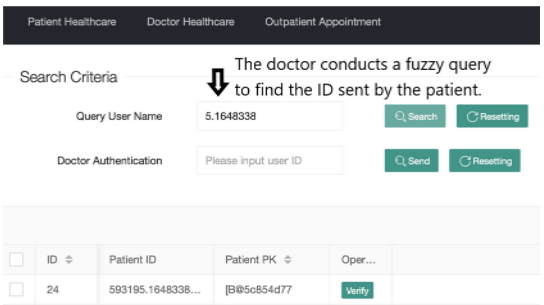

Fig. 5. Doctors inquire about ID from patients.

VII. Conclusion

The security advantage of CLS makes it widely used in health IIoT. In this article, we prove that the existing CLS scheme [21] is vulnerable to the signature attack of the Type-II adversary. We propose a novel CLS scheme and prove it has achieved Girault's level 3 security. The performance analysis shows that our proposal has less computation and communication costs. Hence, we believe that our scheme is suitable for resource-constrained environments. In addition, we utilize our CLS scheme to build a mutual identity authentication and key agreement protocol to confirm the legitimacy of user identity and protect the security of messages in real applications. To sum up, our CLS scheme applies to the application of IIoT technology in interconnected medical devices and sensors.

The emergence of diverse leakage attacks in practical environments has rendered conventional cryptographic primitives previously proven secure under traditional security models unable to maintain their claimed security guarantees. To further enhance the practicality and security of the CLS scheme, the next phase of our research will focus on investigating the leakage resilience of the CLS scheme, aiming to address adversarial scenarios involving partial key or state exposure while preserving its security.

References

[1] P. K. Malik et al., "Industrial Internet of Things and its applications in industry 4.0: State of the art," *Comput. Commun.*, vol. 166, pp. 125–139, Jan. 2021.

[2] J. Li and Y. Zhang, "Cryptanalysis and improvement of batch verification certificateless signature scheme for VANETs," *Wireless Pers. Commun.*, vol. 111, no. 2, pp. 1255–1269, 2020.

[3] Z. Qiao, Q. Yang, Y. Zhou, and M. Zhang, "Improved secure transaction scheme with certificateless cryptographic primitives for IoT-based mobile payments," *IEEE Syst. J.*, vol. 16, no. 2, pp. 1842–1850, Jun. 2022.

[4] J. Li, J. Zhao, and Y. Zhang, "Certificateless online/offline signcryption scheme," *Secur. Commun. Netw.*, vol. 8, no. 11, pp. 1979–1990, 2015.

[5] S. S. Al-Riyami and K. G Paterson, "Certificateless public key cryptography," in *Proc. ASIACRYPT*, 2003, pp. 452–473.

[6] M. Girault, "Self-certified public keys," in *Proc. Workshop Theory Appl. Cryptogr. Techn.*, 1991, pp. 490–497.

[7] Y.-C. Chen, R. Tso, G. Horng, C.-I. Fan, and R.-H. Hsu, "Strongly secure certificateless signature: Cryptanalysis and improvement of two schemes," *J. Inf. Sci. Eng.*, vol. 31, no. 1, pp. 297–314, 2015.

[8] A. Karati, C.-I. Fan, and E.-S. Zhuang, "Reliable data sharing by certificateless encryption supporting keyword search against vulnerable KGC in Industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 18, no. 6, pp. 3661–3669, Jun. 2022.

[9] S. Hussain, S. S. Ullah, I. Ali, J. Xie, and V. N. Inukollu, "Certificateless signature schemes in Industrial Internet of Things: A comparative survey," *Comput. Commun.*, vol. 181, pp. 116–131, Jan. 2022.

[10] Y. Xu et al., "An efficient identity authentication scheme with provable security and anonymity for mobile edge computing," *IEEE Syst. J.*, vol. 17, no. 1, pp. 1012–1023, Mar. 2023.

[11] I. Ali et al., "Certificateless signature-based authentication scheme for vehicle-to-infrastructure communications using bilinear pairing," in *Efficient and Provably Secure Schemes for Vehicular Ad-Hoc Networks*. Singapore: Springer, 2022, pp. 91–119.

[12] C. Wu, H. Huang, K. Zhou, and C. Xu, "Cryptanalysis and improvement of an efficient certificateless signature scheme," *J. Commun. Netw.*, vol. 10, no. 1, pp. 10–17, 2008.

[13] Z. Qiao et al., "An anonymous and efficient certificate-based identity authentication protocol for VANET," *IEEE Internet Things J.*, vol. 11, no. 7, pp. 11232–11245, Apr. 2024.

[14] X. Jia, D. He, Q. Liu, and K.-K. R. Choo, "An efficient provably-secure certificateless signature scheme for Internet-of-Things deployment," *Ad Hoc Netw.*, vol. 71, pp. 78–87, Mar. 2018.

[15] H. Du, Q. Wen, S. Zhang, and M. Gao, "A new provably secure certificateless signature scheme for Internet of Things," *Ad Hoc Netw.*, vol. 100, Apr. 2020, Art. no. 102074.

[16] D. Xiang, X. Li, J. Gao, and X. Zhang, "A secure and efficient certificateless signature scheme for Internet of Things," *Ad Hoc Netw.*, vol. 124, Jan. 2022, Art. no. 102702.

[17] K. Ma et al., "An efficient certificateless signature scheme with provably security and its applications," *IEEE Syst. J.*, vol. 17, no. 4, pp. 5636–5647, Dec. 2023.

[18] L. Wang, K. Chen, Y. Long, and H. Wang, "An efficient pairing-free certificateless signature scheme for resource-limited systems," *Sci. China Inf. Sci.*, vol. 60, no. 11, pp. 1–3, 2017.

[19] G. Thumbur, G. S. Rao, P. V. Reddy, N. B. Gayathri, and D. V. R. K. Reddy, "Efficient pairing-free certificateless signature scheme for secure communication in resource-constrained devices," *IEEE Commun. Lett.*, vol. 24, no. 8, pp. 1641–1645, Aug. 2020.

[20] Z. Xu, M. Luo, M. K. Khan, K.-K. R. Choo, and D. He, "Analysis and improvement of a certificateless signature scheme for resource-constrained scenarios," *IEEE Commun. Lett.*, vol. 25, no. 4, pp. 1074–1078, Apr. 2020.

[21] W. Wang, H. Xu, M. Alazab, T. R. Gadekallu, Z. Han, and C. Su, "Blockchain-based reliable and efficient certificateless signature for IIoT devices," *IEEE Trans. Ind. Informat.*, vol. 18, no. 10, pp. 7059–7067, Oct. 2022.

[22] S. Liu, Z. Wan, Y. Yuan, Q. Dong, B. Yang, and F. Hao, "An efficient certificateless blind signature scheme with conditional revocation for mobile crowd sensing within smart city," *IEEE Internet Things J.*, vol. 11, no. 9, pp. 15985–15997, May 2024.

[23] K.-A. Shim, "A secure certificateless signature scheme for cloud-assisted Industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 20, no. 4, pp. 6834–6843, Apr. 2024.

[24] K.-A. Shim, "Design principles of secure certificateless signature and aggregate signature schemes for IoT environments," *IEEE Access*, vol. 10, pp. 124848–124857, 2022.

[25] W. Yang, S. Wang, and Y. Mu, "An enhanced certificateless aggregate signature without pairings for e-healthcare system," *IEEE Internet Things J.*, vol. 8, no. 6, pp. 5000–5008, Mar. 2021.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 14,2025 at 03:49:37 UTC from IEEE Xplore. Restrictions apply.

[26] Y. Liang and Y. Liu, "Analysis and improvement of an efficient certificateless aggregate signature with conditional privacy preservation in VANETs," *IEEE Syst. J.*, vol. 17, no. 1, pp. 664–672, Mar. 2023.

[27] W. Xiong, R. Wang, Y. Wang, Y. Wei, F. Zhou, and X. Luo, "Improved certificateless aggregate signature scheme against collusion attacks for VANETs," *IEEE Syst. J.*, vol. 17, no. 1, pp. 1098–1109, Mar. 2023.

[28] V. K. Yadav, N. Andola, S. Verma, and S. Venkatesan, "PSCLS: Provably secure certificateless signature scheme for IoT device on cloud," *J. Supercomput.*, vol. 79, no. 5, pp. 4962–4982, 2023.

[29] B. C. Hu, D. S. Wong, Z. Zhang, and X. Deng, "Key replacement attack against a generic construction of certificateless signature," in *Proc. Australas. Conf. Inf. Security Privacy*, 2006, pp. 235–246.

[30] D. Pointcheval and J. Stern, "Security arguments for digital signatures and blind signatures," *J. Cryptol.*, vol. 13, pp. 361–396, Jun. 2000.

[31] Y. Jiang, K. Zhang, Y. Qian, and L. Zhou, "Anonymous and efficient authentication scheme for privacy-preserving distributed learning," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2227–2240, 2022.

[32] M. Rasslan, M. M. Nasreldin, and H. K. Aslan, "Ibn Sina: A patient privacy-preserving authentication protocol in medical Internet of Things," *Comput. Secur.*, vol. 119, 2022, Art. no. 102753.

[33] I. Alam and M. Kumar, "A novel authentication protocol to ensure confidentiality among the Internet of Medical Things in COVID-19 and future pandemic scenario," *Internet Things*, vol. 22, Jul. 2023, Art. no. 100797.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on August 14,2025 at 03:49:37 UTC from IEEE Xplore. Restrictions apply.